Testiranje serverskog dela Spring veb aplikacije



Test runners



- Testiranje Spring aplikacije koristi JUnit
- JUnit predviđa različite *Test runners*
- *Test runner*
  - komponenta zadužena za izvršavanje testa
  - instancira objekat koji sadrži @*Test* metode, izvršava metode i kreira izveštaj o testu
- JUnit ima ugrađeni konzolni *test runner*
- Eclipse sadrži svoj grafički *test runner*
- Moguće je koristiti proizvoljan *test runner*

Test runners

- Od JUnit5, anotacijom @SpringBootTest vrši se inicijalizacija Spring kontejnera pri pokretanju testova, kako bi bili dostupni svi objekti kojima Spring aplikacija upravlja
- Ovo će automatski u JUnit uključiti SpringExtension komponentu koja omogućuje testiranje specifično za Spring

Testiranje spring veb aplikacije

- Kreiraju se testovi koji verifikuju funkcionalnosti serverskih metoda
- Ne postoji generalno pravilo koje tipove testova realizovati i kako
- Jedna varijanta su izolovani jedinični testovi za svaki sloj aplikacije
  - potrebno je *mock* objektima simulirati funkcionalnosti drugih slojeva
  - u praksi je često preveliki trošak testiranja svakog sloja pojedinačno

Testiranje spring veb aplikacije

- Druga varijanta je odmah kreirati integracione testove
  - testiraju više slojeva aplikacije odjednom
- Dve varijante
  - svi slojevi se testiraju jednom grupom testova ili
  - testiranje grupa slojeva se razdvaja u više grupa integracionih testova

Test podaci

- Prva varijanta je da sami testovi inicijalizuju test podatke koji su im potrebni
- Nedostaci ovog pristupa
  - komplikovano testiranje većih entiteta koji koriste više povezanih objekata
    - iznova bi različiti testovi morali da ponavljaju inicijalizaciju podataka
  - komplikovano je inicijalizovati veću količinu podataka što je neophodno za testiranje rada u produkcionim uslovima
- Druga varijanta je da se korišćenjem skripte baza podataka inicijalizuje test podacima
  - testovi polaze od toga da određeni podaci postoje i verifikuju ponašanje aplikacije u odnosu na te podatke

Spring Boot
Testiranje JPA sloja

- Možemo koristiti realnu bazu
- Druga varijanta je korišćenje testne in-memory baze
  - Postavlja se anotacija @DataJpaTest
  - Koja konkretno baza podataka će biti korišćena zavisi od biblioteke koja je dodata u CLASSPATH aplikacije
    - tipično H2
    - ako koristimo Maven, dodamo bazu u *dependencies*
  - Koristimo *TestEntityManager* za pristup perzistentnom kontekstu

```
@Autowired
private TestEntityManager entityManager;
```

Spring Boot
Testiranje servisa

- Možemo praviti integracioni test koji se stvarno obraća realnim objektima iz drugih slojeva
- Druga varijanta je izolovani jedinični test
  - postavljamo test dvojnike na mesto objekata iz drugih slojeva

```
@MockBean
CountryRepository countryRepository;

@Before
public void setup() {
 given(countryRepository.findOne(1L)).willReturn(
   new Country(1L, "Serbia", 7098000));
}
```

Spring Boot
Testiranje kontrolera

- Klasa *TestRestTemplate* omogućuje obraćanje REST servisu

```
@Autowired
private TestRestTemplate restTemplate;

@Test
public void testGetCountries() {
   ResponseEntity<CountryDTO[]> re =
      restTemplate.getForEntity("/api/countries",
           CountryDTO[].class);
   CountryDTO[] countries = responseEntity.getBody();
   assertEquals(HttpStatus.OK, re.getStatusCode());
   assertEquals(3, countries.length);
   …
```

Spring Boot
Testiranje kontrolera

- Alternativa klasi *TestRestTemplate* je korišćenje klase *MockMvc*

- Ova klasa testira na nivou samog JSON sadržaja koji se razmenjuje sa kontrolerom
  - Za razliku od *TestRestTemplate* koja posmatra deserijalizovane Java objekte

Spring Boot
Primer testova



- Primeri
  - `TestCountry`
  - `TestSecurityCountry`